linalg: A Dense Numerical Linear Algebra Library in C++17

Aren Vista, Micah Havens, Sina Roomi, Max Monson, Jingson Guo, Dominic Mello

August 1, 2026

Contents

This project is a C++ library for numerical linear algebra. It provides a set of classes for performing operations on dense matrices and vectors — arithmetic, norms, factorization, linear solving, least squares, and eigenvalue computation — implemented from first principles rather than by wrapping BLAS/LAPACK. The library is designed to be efficient and easy to use, with a focus on exploring the techniques commonly taught in numerical linear algebra, and to serve as a demonstration of ability to design and build robust numerical software: choosing algorithms for stability rather than convenience, handling floating-point edge cases (overflow-safe norms, cancellation-free reflectors, deflation criteria), and verifying every component against the mathematical identities it must satisfy.

Complementary to the library is a set of notes that explain the algorithms and the rationale behind their computational methods. These notes can be accessed here: Numerical Linear Algebra — L01: Basic Matrix Algebra.

1 What the Library Covers

- **Core types.** Owning `Matrix` / `Vector` classes over a scalar abstraction (`NumericTraits`) that supports `float`, `double`, and their complex counterparts uniformly, plus non-owning stride-aware views (`MatrixView`) so blocked algorithms can operate on panels without copying.
- **Computational kernels.** BLAS-shaped level 1/2/3 routines (`gemm`, `gemv`, `trsm`, …) that serve as the single tuning surface for performance work, and the two workhorse orthogonal primitives — Householder reflectors and Givens rotations — stored implicitly and applied in $O(mn)$.
- **Direct factorizations.** LU with partial and full pivoting, Cholesky and LDL^H, Householder QR with and without column pivoting, and the condensed forms (Hessenberg, tridiagonal, bidiagonal) that feed the eigenvalue and singular value algorithms.
- **Eigenvalues and the SVD.** Symmetric eigensolving via implicit-shift QR with Wilkinson shifts, the real Schur form by Francis double-shift sweeps, general eigenvectors on top of it, and the Golub–Kahan SVD.
- **High-level solvers.** A dispatching `LinearSolver` that inspects the matrix and chooses a factorization, least squares by QR/SVD (with the normal equations included only as a cautionary comparison), and Gauss–Newton / Levenberg–Marquardt for nonlinear problems.

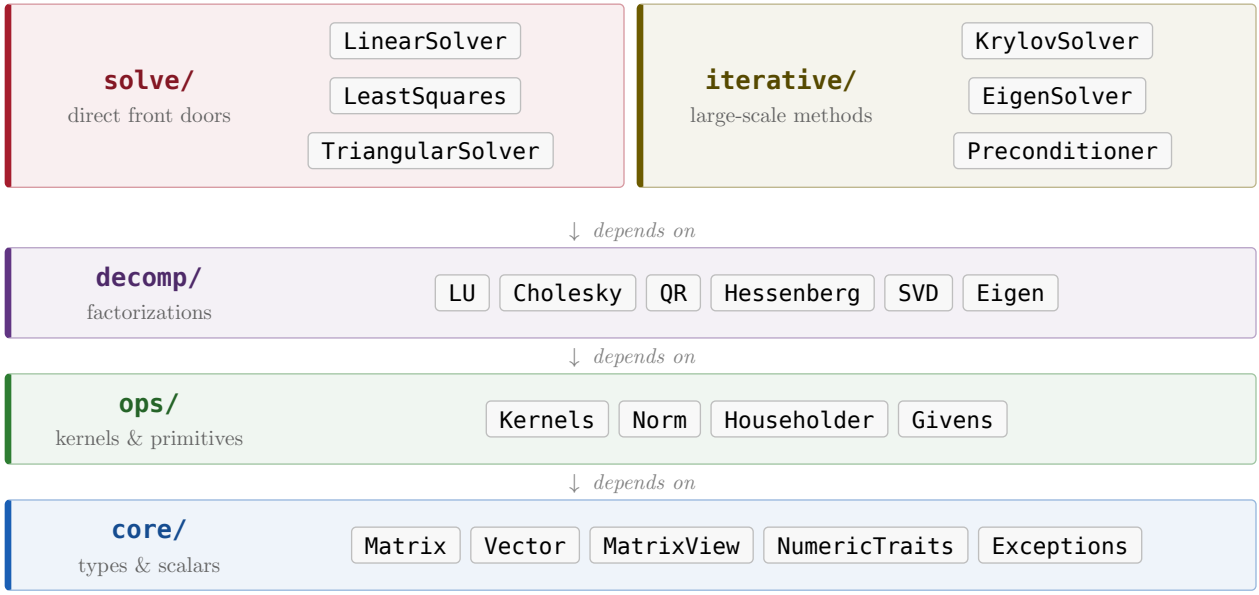

Figure 1: The dependency tiers of the library. Each tier is written only in terms of the tiers below it, so tuning the `ops` kernels accelerates every factorization and solver above without touching their code.

1.1 The Kernels as the Tuning Surface

"BLAS-shaped" means the kernels follow the interface conventions of the Basic Linear Algebra Subprograms, the de facto standard vocabulary of dense computation since the 1970s: strided vector arguments, an `alpha` / `beta` scaling pair, and in-place updates like $C \leftarrow \alpha AB + \beta C$ (`gemm`) rather than functions that allocate and return results. The *levels* classify routines by the ratio of arithmetic to data touched:

- **Level 1** — vector–vector, $O(n)$ work on $O(n)$ data: `axpy`, `dot`, `nrm2`. Memory-bound; there is little to tune.
- **Level 2** — matrix–vector, $O(n^2)$ work on $O(n^2)$ data: `gemv`, `ger`, `trsv`. Still memory-bound: every matrix element is read once and used once.
- **Level 3** — matrix–matrix, $O(n^3)$ work on $O(n^2)$ data: `gemm`, `syrk`, `trsm`. The only level with real data reuse — each element participates in $\sim n$ operations — which is what cache blocking, packing, and SIMD micro-kernels can exploit to approach the hardware's peak floating-point throughput.

This is why the kernels are the *single tuning surface*: the factorizations above are written as thin orchestration around kernel calls (a blocked LU is a panel factorization plus `trsm` and `gemm` updates on the trailing submatrix). Optimizing `gemm` alone therefore accelerates LU, Cholesky, QR, and the condensed-form reductions at once, with no change to their code — mirroring how LAPACK gets its speed from whatever tuned BLAS it is linked against. It also bounds the risk of performance work: the naive triple-loop kernels stay as the reference implementation, and a tuned variant must merely reproduce their results to be correct everywhere.

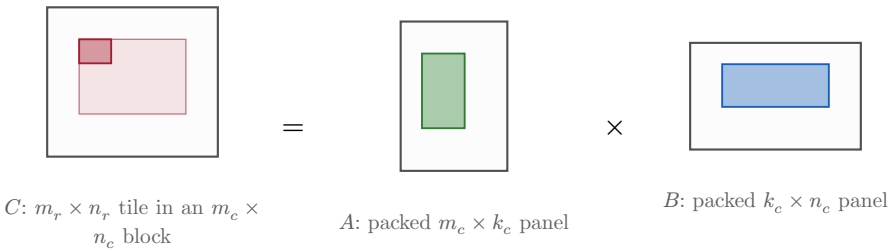

Figure 2: Level-3 blocking: `gemm` packs a panel of A and a panel of B into contiguous buffers sized for the cache hierarchy, then a register-tiled micro-kernel sweeps small tiles of C, reusing every packed element $\sim n$ times.

2 A Worked Kernel: Givens Rotations

The file `src/ops/Givens.cpp` is a good place to see the library's guiding principle — *choose the algorithm for numerical stability, then let the code mirror the math it implements* — at the scale of a single kernel. A Givens rotation is the second of the two orthogonal primitives (the other being the Householder reflector); it is the tool of choice when only one entry must be zeroed at a time, as in bidiagonal sweeps, QR updating, and the Hessenberg reduction inside GMRES.

Definition (Givens rotation) . A *Givens rotation* acting on the plane spanned by coordinates $p < q$ is the identity everywhere except in that 2×2 block, where it is

$$G = \begin{bmatrix} c & s \\ -\overline{s} & c \end{bmatrix}, \quad c \in \mathbb{R}, \quad c^2 + |s|^2 = 1.$$

Keeping c real while allowing $s \in \mathbb{C}$ makes G unitary for complex scalars too, so the same object serves the real and complex builds.

Because G differs from the identity in only two rows and two columns, the class never stores the dense matrix: the members are just `cosine_` (c), `sine_` (s), the plane indices `p_`, `q_`, and a

cached `radius_` (explained below). Applying it is therefore $O(mn)$, not $O(mn^2)$ — the implicit representation *is* the performance story.

2.1 Constructing the rotation

The core routine `FromPair(a, b, p, q)` chooses (c, s) so that the rotation maps the pair (a, b) onto $(r, 0)$, zeroing the second component:

$$\begin{bmatrix} c & s \\ -\overline{s} & c \end{bmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} r \\ 0 \end{pmatrix}.$$

Theorem (closed form) . With $d = \sqrt{|a|^2 + |b|^2}$, the solution is

$$c = \frac{|a|}{d}, \quad s = \frac{a}{|a|} \cdot \frac{\overline{b}}{d}, \quad r = \frac{a}{|a|} \cdot d.$$

One checks directly that c is real, $c^2 + |s|^2 = 1$, and the lower entry $-\overline{s}a + cb = 0$.

Translating the closed form into code line-for-line would be a mistake: forming $|a|^2$ overflows for $a \approx 10^{200}$ in double precision and underflows to zero for $a \approx 10^{-200}$, even though the true d is perfectly representable in both cases. The implementation follows LAPACK's `dlartg` and never squares the inputs directly. It scales by the larger magnitude first,

$$\sigma = \max(|a|, |b|), \quad d = \sigma \sqrt{\left(\frac{|a|}{\sigma}\right)^2 + \left(\frac{|b|}{\sigma}\right)^2},$$

so the squared terms live in $[0, 1]$ and the lone $\sqrt{\cdot}$ cannot over- or underflow. The code names these quantities exactly: `magA`, `magB` are $|a|, |b|$; `scale` is σ; `scaledNorm` is the bracketed root; `norm` is d; `phase` is the unit-modulus direction $a/|a|$; and the results land in `cosCoeff`, `sinCoeff`, and `radius`. Two exact-zero branches handle the degenerate planes: $b = 0$ needs no rotation (identity, with $r = a$ carrying a's sign), and $a = 0$ is a pure swap.

Remark (Why cache the radius) . `radius_` stores the r that (a, b) lands on. It is exactly the new diagonal entry a factorization writes after the rotation zeroes the subdiagonal one, so caching it during construction saves recomputing $|\cdot|$ at the call site. The accessor `radius()` simply returns it; the constructors set it to zero since a rotation built from raw coefficients has no associated pair.

2.2 Applying and undoing the rotation

The three `apply*` methods are the same 2×2 update specialized to different targets. Left application `applyLeft` rotates two *rows*,

$$\text{row}_p \leftarrow c \cdot \text{row}_p + s \cdot \text{row}_q, \quad \text{row}_q \leftarrow -\overline{s} \cdot \text{row}_p + c \cdot \text{row}_q,$$

`apply` is this update on a single vector, and `applyRight` is the right-multiplication MG, which rotates two *columns* with the transposed coefficient pattern. Each loop reads both old values before writing either, so the second assignment never sees a value the first one already overwrote.

Finally, `transposed` and `inverse` look identical for real scalars but are deliberately distinct for complex ones, which is where the "c real, s complex" convention earns its keep:

$$G^T = \begin{bmatrix} c & -\overline{s} \\ s & c \end{bmatrix} \quad (\text{store } c, -\overline{s}), \qquad G^{-1} = G^H = \begin{bmatrix} c & -s \\ \overline{s} & c \end{bmatrix} \quad (\text{store } c, -s).$$

Since G is unitary, its inverse is the conjugate transpose G^H, so `inverse` negates s while `transposed` negates $\overline{s}$. For real s the two coincide, matching the familiar fact that a real rotation's inverse is its transpose — but using `transposed` where `inverse` is meant would silently give a non-unitary "undo" on complex data.

2.3 Method Reference

`Givens<T>` — a single plane rotation on coordinates (p, q):

Method	Description	Example
`Givens()`	Default constructor: an uninitialized rotation ($c = s = 0$, radius 0).	`Givens<double> g;`
`Givens(c, s, p, q)`	Construct directly from coefficients and plane indices.	`Givens<double>(0.6, 0.8, 0, 1)`
`FromPair(a, b, p, q)`	*(static)* The rotation carrying (a, b) to $(r, 0)$; overflow-safe, `dlartg`-style.	`Givens<double>::FromPair(3.0, 4.0, 0, 1)`
`Identity(p, q)`	*(static)* The no-op rotation $c = 1, s = 0$ on the plane (p, q).	`Givens<double>::Identity(0, 1)`
`cosine()`	The real cosine coefficient c.	`g.cosine()` $\to 0.6$
`sine()`	The sine coefficient s.	`g.sine()` $\to 0.8$
`firstIndex()`	The lower plane index p.	`g.firstIndex()` $\to 0$
`secondIndex()`	The upper plane index q.	`g.secondIndex()` $\to 1$
`radius()`	The r that `FromPair` mapped (a, b) onto.	`g.radius()` $\to 5.0$
`transposed()`	The transpose G^T (stored as $c, -\overline{s}$).	`g.transposed()`
`inverse()`	The inverse $G^{-1} = G^H$ (stored as $c, -s$); unitary.	`g.inverse()`
`applyLeft(view)`	In place: overwrites `target` with $G \cdot$ `target`, rotating rows p, q. O(cols).	`g.applyLeft(A.view())`
`applyRight(view)`	In place: overwrites `target` with `target` $\cdot\, G$, rotating columns p, q. O(rows).	`g.applyRight(A.view())`
`apply(x)`	In place rotation of vector entries p and q.	`g.apply(x)`
`toMatrix(n)`	The dense $n \times n$ rotation embedded in the identity (tests only).	`g.toMatrix(2)`

`GivensSequence<T>` — an ordered product $G_{k-1} \ldots G_1 G_0$ of rotations:

Method	Description	Example
`GivensSequence()`	Construct an empty sequence (count = 0).	`GivensSequence<double> seq;`
`append(g)`	Add a rotation to the end of the sequence.	`seq.append(g);`
`clear()`	Remove all rotations.	`seq.clear();`
`count()`	The number of rotations held.	`seq.count()`

Method	Description	Example
`operator[](k)`	Read-only access to rotation k (append order).	`seq[0]`
`applyLeft(view)`	Apply all on the left in append order: `target` $\leftarrow G_{k-1}...G_0 \cdot$ `target` .	`seq.applyLeft(A.view())`
`applyRight(view)`	Apply all on the right in append order.	`seq.applyRight(A.view())`
`applyLeftReversed(view)`	Apply all on the left in *reverse* append order (used by the inverse sweep).	`seq.applyLeftReversed(A.view())`
`reversed()`	A copy of the sequence in reverse order.	`seq.reversed()`
`toMatrix(n)`	The dense $n \times n$ accumulated product.	`seq.toMatrix(3)`

2.4 Example: one step of Givens QR

A Givens rotation triangularizes a matrix one subdiagonal entry at a time. The snippet below zeroes $A_{1,0}$ of a 2×2 matrix and accumulates the rotation so the orthogonal factor can be recovered — the atom from which a full Givens QR is built by looping over the subdiagonal.

```
#include "linalg/core/Matrix.hpp"
#include "linalg/ops/Givens.hpp"
using namespace linalg;

Matrix<double> A{{4.0, 3.0},
                 {3.0, 2.0}};

// Choose the rotation on rows (0, 1) that sends (A(0,0), A(1,0)) to (r, 0).
Givens<double> g = Givens<double>::FromPair(A(0, 0), A(1, 0), 0, 1);

g.applyLeft(A.view());   // A <- G * A; now A(1,0) == 0 and A(0,0) == g.radius()

// Collect rotations so Q^T = ... G_1 G_0 can be materialized or reapplied.
GivensSequence<double> qt;
qt.append(g);
Matrix<double> Qt = qt.toMatrix(2);   // the accumulated orthogonal factor

// A single vector rotates the same way:
Vector<double> x{4.0, 3.0};
g.apply(x);              // x(1) == 0, x(0) == g.radius()
```

After `applyLeft`, `A` is upper-triangular and its new diagonal entry equals `g.radius()` — the value `FromPair` cached precisely because it is the entry a QR step writes next. Extending the loop to larger matrices (zeroing $A_{i,0}$ for $i = m-1, ..., 1$, then moving to the next column) reduces A to R while `qt` accumulates Q^H.

3 Module Reference

The two lowest tiers of the dependency graph — `core/` (the types every other file is written against) and `ops/` (the kernels and orthogonal primitives built directly on them) — are the foundation the factorizations and solvers orchestrate. Each header declares; the matching `src/` file defines and explicitly instantiates for the supported scalar list.

3.1 Core Types (`core/`)

`Traits` — the scalar abstraction

`NumericTraits<T>` is the single place the library asks questions about a scalar instead of assuming `double`, and the reason one template body serves `float`, `double`, and their `complex` counterparts. It draws the line between a scalar `T` and its magnitude type `Real` (equal for real `T`, the component type for `complex`). All members are `static`.

Member	Description	Example
QUERIES		
`epsilon()`	Machine epsilon of `Real` — the gap between 1 and the next value.	`epsilon()` $\approx 2.2 \times 10^{-16}$ for `double`
`safeMin()`	Smallest `s` with $1/s$ finite; the scaling threshold for overflow-safe code.	`safeMin()` $\approx 2.2 \times 10^{-308}$
ARITHMETIC		
`abs(x)`	Magnitude $\lvert x \rvert$ as a `Real`.	`abs({3.0, 4.0})` $\to 5.0$
`absSquared(x)`	$\lvert x \rvert^2$, computed without a square root.	`absSquared({3.0, 4.0})` $\to 25.0$
`conj(x)`	Complex conjugate (the identity for real `T`).	`conj({3.0, 4.0})` $\to (3, -4)$
`real(x)` / `imag(x)`	Real and imaginary parts (imag is 0 for real `T`).	`imag({3.0, 4.0})` $\to 4.0$
`sqrt(x)`	Principal square root.	`sqrt(2.0)` ≈ 1.414
`zero()` / `one()`	The additive / multiplicative identity for `T`.	`one()` $\to (1, 0)$ for `complex`
`isApproxZero(x, tol)`	Whether $\lvert x \rvert \le$ tol.	`isApproxZero(1e-20, 1e-9)` $\to$ `true`

The same header carries `IsComplex<T>::value` (the compile-time category test) and the tag enums — `StorageOrder`, `Triangle`, `Transposition`, `Diagonal` — that kernels and solvers select behavior with.

```
using T = std::complex<double>;
using R = NumericTraits<T>::Real;              // double
R m  = NumericTraits<T>::abs({3.0, 4.0});      // 5.0
T zc = NumericTraits<T>::conj({3.0, 4.0});     // (3, -4)
constexpr bool cplx = IsComplex<T>::value;     // true
```

`Exceptions` — the error hierarchy

Everything derives from `LinalgError` (itself a `std::runtime_error`), so a single `catch (const LinalgError&)` handles the whole family; the typed subclasses carry structured context for callers that want it.

Type	Meaning and payload	Example
`LinalgError`	Base type; thrown by every unimplemented stub and by generic contract violations.	`throw LinalgError("not implemented: ...")`
`DimensionMismatch`	Incompatible operand shapes; `lhsRows()`, `lhsCols()`, `rhsRows()`, `rhsCols()`.	Adding a 2×3 to a 3×2 matrix.
`IndexOutOfRange`	Bad index from a checked `at()`; `index()`, `bound()`.	`v.at(9)` on a length-3 vector.
`SingularMatrix`	A (near-)zero pivot in a factorization or solve; `pivotIndex()`.	`Matrix::Zeros(3,3).inverse()`

Type	Meaning and payload	Example
`NotPositiveDefinite`	First non-positive pivot in a Cholesky-family factorization; `pivotIndex()`.	Cholesky of an indefinite matrix.
`ConvergenceFailure`	An iterative method exhausted its budget; `algorithm()`, `iterations()`.	GMRES that never meets its tolerance.
`NotComputed`	A factorization accessor called before `compute()`, or after one that failed.	`lu.solve(b)` before `lu.compute(A)`.

```cpp
try {
    Matrix<double> Ainv = A.inverse();          // may throw
} catch (const SingularMatrix& e) {
    std::cerr << "zero pivot at step " << e.pivotIndex() << "\n";
} catch (const LinalgError& e) {                // base catches the rest
    std::cerr << e.what() << "\n";
}
```

`Vector` — owning dense vector

Kept a distinct type from an $n \times 1$ `Matrix` precisely so that `dot`, `outer`, and `norm` have one unambiguous meaning rather than a row-versus-column ambiguity. Arithmetic operators are members only.

Member	Description	Example
CONSTRUCTION		
`Vector()`	Empty, length 0.	`Vector<double> v;`
`Vector(n)` / `Vector(n, fill)`	`n` zeros, or `n` copies of `fill`.	`Vector<double>(3, 1.0)` → $(1, 1, 1)$
`Vector({...})` / `Vector(std::vector)`	From a braced list or a `std::vector`.	`Vector<double>{3.0, 4.0}`
copy/move, `~Vector`, `operator=`	Standard deep-copy value semantics.	`Vector<double> w = v;`
`Zeros` / `Ones` / `Constant`	*(static)* Filled vectors of a given length.	`Vector<double>::Ones(3)`
`Unit(n, axis)`	*(static)* The basis vector e_{axis}.	`Unit(3, 0)` → $(1, 0, 0)$
`Random(n, seed)`	*(static)* Reproducible pseudo-random entries.	`Vector<double>::Random(4, 42)`
`LinSpace(n, begin, end)`	*(static)* Equally spaced values, endpoints inclusive.	`LinSpace(3, 0.0, 1.0)` → $(0, .5, 1)$
ELEMENT ACCESS		
`operator()(i)` / `operator[](i)`	Unchecked element access (mutable and const).	`v(0) = 5.0;`
`at(i)`	Bounds-checked access; throws `IndexOutOfRange`.	`v.at(0)`
`data()`	Raw buffer pointer.	`v.data()`
`size()` / `isEmpty()`	Length, and whether it is 0.	`v.size()` → 2
ARITHMETIC		
`+` `-` `*` `/` (unary `-`)	Elementwise add/sub and scalar mul/div; negation.	`v * 2.0`
`+=` `-=` `*=` `/=`	In-place forms.	`v += w;`
`==` / `!=`	Exact equality / inequality.	`v == w`

Member	Description	Example
`scaledBy(s)`	Scalar multiple (readable mirror of `operator*`).	`v.scaledBy(2.0)`
`elementwiseProduct` / `elementwiseQuotient`	Hadamard product / quotient.	`v.elementwiseProduct(w)`
PRODUCTS		
`dot(y)`	Unconjugated bilinear form $\sum x_i y_i$.	`x.dot(y)` → 11.0
`hermitianDot(y)`	Hermitian inner product $\sum \overline{x_i} y_i$ (conjugates `*this`).	`x.hermitianDot(y)`
`outer(y)`	Outer product xy^H as a matrix.	`x.outer(y)` → matrix
`cross(y)`	Cross product (size-3 vectors only).	`e1.cross(e2)` → e_3
`axpy(alpha, y)`	Fused $\alpha x + y$.	`x.axpy(2.0, y)`
NORMS & SCALAR SUMMARIES		
`norm()`	Euclidean 2-norm, overflow-safe scaled.	`x.norm()` → 5.0
`squaredNorm()`	$\sum \lvert x_i \rvert^2$ (unscaled; may overflow).	`x.squaredNorm()` → 25.0
`oneNorm()` / `infinityNorm()` / `pNorm(p)`	The 1-, ∞-, and general p-norms.	`x.infinityNorm()` → 4.0
`sum()` / `product()`	Sum and product of all elements.	`x.sum()` → 7.0
`normalized()` / `normalize()`	Unit-norm copy / in-place; throw on a zero vector.	`x.normalized()` → $(.6, .8)$
SHAPE		
`segment(start, count)` / `head` / `tail`	Contiguous sub-vectors.	`v.head(2)`
`concat(y)` / `reversed()`	Concatenation; reverse order.	`v.reversed()`
`asColumnMatrix` / `asRowMatrix` / `asDiagonalMatrix`	Reinterpret as a matrix.	`v.asDiagonalMatrix()`
`resize` / `conservativeResize`	Resize, discarding or preserving overlap.	`v.resize(5)`
`fill` / `setZero` / `setUnit(axis)` / `swap`	In-place mutation.	`v.setZero();`
REDUCTIONS & DIAGNOSTICS		
`maxAbsIndex` / `minAbsIndex`	Index of largest / smallest magnitude (ties: lowest index).	`x.maxAbsIndex()` → 1
`maxCoefficient` / `minCoefficient`	Largest / smallest coefficient (by real part for complex `T`).	`x.maxCoefficient()` → 4.0
`isApprox(y, tol)` / `hasNaN()`	Approximate equality; NaN check.	`x.isApprox(y, 1e-9)`
`toString(precision)`	Human-readable formatting.	`x.toString(3)`

```
Vector<double> x{3.0, 4.0};
double n = x.norm();                  // 5.0 (overflow-safe)
Vector<double> u = x.normalized();    // (0.6, 0.8)
Vector<double> y{1.0, 2.0};
double d = x.dot(y);                  // 11.0
Vector<double> z = x.axpy(2.0, y);    // 2*x + y = (7, 10)
```

`Matrix` — owning dense matrix

Row-major and contiguous. Every operator is a member, so `matrix * scalar` exists but `scalar * matrix` does not (use `scaledBy()`) — a deliberate asymmetry that keeps overload resolution unsurprising. The `O(n^3)` convenience summaries (`determinant`, `spectralNorm`, `conditionNumber`, `rank`, `inverse`, `pseudoInverse`) factorize internally; hold a decomposition object instead when you need more than one query.

Member	Description	Example
CONSTRUCTION		
`Matrix()`	Empty 0×0.	`Matrix<double> A;`
`Matrix(r, c[, fill])`	Shape $r \times c$, zero- or `fill`-initialized.	`Matrix<double>(2, 2, 1.0)`
`Matrix(r, c, rowMajorData)`	From a flat row-major `std::vector`.	`Matrix<double>(2, 2, {1,2,3,4})`
`Matrix({{...}})`	From a braced list of equal-length rows.	`Matrix<double>{{4,1}, {1,3}}`
`Matrix(ConstMatrixView)`	Deep-copy a view (explicit).	`Matrix<double>(A.block(0,0,2,2))`
copy/move, `~Matrix`, `operator=`	Standard deep-copy value semantics.	`Matrix<double> B = A;`
`Zeros` / `Ones` / `Constant`	*(static)* Filled matrices.	`Matrix<double>::Zeros(2, 3)`
`Identity(n)` / `Diagonal(v)`	*(static)* Identity; diagonal from a vector.	`Matrix<double>::Identity(3)`
`Random` / `RandomSymmetric` / `RandomOrthogonal`	*(static)* Reproducible random, Hermitian, or unitary matrices.	`Matrix<double>::RandomOrthogonal(4, 7)`
`Hilbert(n)` / `Vandermonde(nodes, deg)`	*(static)* Classic ill-conditioned / structured inputs.	`Matrix<double>::Hilbert(5)`
`FromColumns` / `FromRows`	*(static)* Assemble from vectors.	`Matrix<double>::FromColumns({u, v})`
ELEMENT ACCESS & SHAPE QUERIES		
`operator()(i, j)` / `at(i, j)`	Unchecked / bounds-checked element access.	`A(0, 1) = 5.0;`
`data()`	Row-major buffer; (i, j) at `data()[i*cols()+j]`.	`A.data()`
`rows()` / `cols()` / `size()`	Dimensions and element count.	`A.rows()` $\to 2$
`isEmpty()` / `isSquare()`	Shape predicates.	`A.isSquare()` $\to$ `true`
VIEWS & ROW/COL ACCESS		
`view()` / `block(...)`	Mutable and const aliasing views (whole or sub-block).	`A.block(0, 0, 2, 2)`
`rowView(i)` / `colView(j)`	Aliasing views of a single row / column.	`A.colView(0)`
`row(i)` / `col(j)` / `diagonal()`	Owning **copies** into a `Vector`.	`A.diagonal()`
`setRow` / `setCol` / `setBlock`	Overwrite a row, column, or block.	`A.setRow(0, v)`
ARITHMETIC		

Member	Description	Example
`+` `-` `*` `/` (unary `-`)	Matrix ±; matrix×matrix, matrix×vector, matrix×scalar; scalar div; negation.	`A * B`, `A * x`, `A * 2.0`
`+=` `-=` `*=` `/=`	In-place forms.	`A += B;`
`==` / `!=`	Exact equality / inequality.	`A == B`
`scaledBy(s)`	sA (scalar on the left).	`A.scaledBy(2.0)`
`elementwiseProduct` / `elementwiseQuotient`	Hadamard product / quotient.	`A.elementwiseProduct(B)`
`kroneckerProduct(B)`	Kronecker (tensor) product.	`A.kroneckerProduct(B)`
`power(k)`	Integer matrix power ($k = 0$ gives I).	`A.power(3)`
SHAPE MANIPULATION		
`transpose()` / `conjugateTranspose()`	A^T / A^H (equal for real `T`).	`A.transpose()`
`reshaped(r, c)`	Reinterpret row-major with a new shape.	`A.reshaped(1, 4)`
`horizontalConcat` / `verticalConcat`	Stack side by side / on top.	`A.horizontalConcat(B)`
`withoutRow(i)` / `withoutCol(j)`	Copy with one row / column dropped.	`A.withoutRow(0)`
`transposeInPlace()`	In-place transpose.	`A.transposeInPlace();`
`resize` / `conservativeResize`	Resize, discarding or preserving the top-left block.	`A.conservativeResize(3, 3)`
`swapRows` / `swapCols`	In-place row / column swap.	`A.swapRows(0, 1);`
`fill` / `setZero` / `setIdentity` / `swap`	In-place mutation.	`A.setIdentity();`
SCALAR SUMMARIES		
`trace()` / `sum()`	Diagonal sum; sum of all elements.	`A.trace()` → 7.0
`determinant()`	Determinant via LU.	`A.determinant()` → 11.0
`oneNorm` / `infinityNorm` / `frobeniusNorm` / `maxNorm`	Entrywise / induced norms (no factorization).	`A.frobeniusNorm()`
`spectralNorm` / `conditionNumber` / `rank(tol)`	SVD-based, $O(n^3)$.	`A.conditionNumber()`
PREDICATES		
`isSymmetric` / `isHermitian` / `isDiagonal`	Structure tests within an absolute tolerance.	`A.isSymmetric(0.0)`
`isTriangular(which, tol)` / `isOrthogonal(tol)`	Triangularity; $A^H A = I$.	`A.isOrthogonal(1e-12)`
`isApprox(B, tol)` / `hasNaN()`	Approximate equality; NaN check.	`A.isApprox(B, 1e-9)`
DERIVED MATRICES & SERIALIZATION		
`inverse()`	Inverse via LU; throws `SingularMatrix`.	`A.inverse()`
`pseudoInverse(tol)`	Moore–Penrose pseudoinverse via SVD.	`A.pseudoInverse(1e-12)`
`triangularPart(which)`	Keep one triangle, zero the other.	`A.triangularPart(Triangle::Kind::Upper)`
`symmetricPart()` / `skewSymmetricPart()`	$(A \pm A^H)/2$.	`A.symmetricPart()`
`toString` / `toMatlabLiteral`	Human-readable / MATLAB-literal formatting.	`A.toMatlabLiteral()`
`FromCsv(path)` / `writeCsv(path)`	CSV read *(static)* / write.	`A.writeCsv("A.csv")`

```cpp
Matrix<double> A{{4.0, 1.0},
                 {1.0, 3.0}};
double det = A.determinant();          // 11 (via LU)
Matrix<double> Ainv = A.inverse();     // via LU
bool sym   = A.isSymmetric(0.0);       // true
Matrix<double> P = A * Ainv;           // ~ Identity(2)
```

`MatrixView` / `ConstMatrixView` — non-owning windows

Element (i, j) lives at `data[i * rowStride + j * colStride]`, so rows, columns, diagonals, transposes, and rectangular blocks are all expressible as views over the same storage — which is what lets blocked algorithms operate on panels without copying. Copy assignment **rebinds** the view; `copyFrom` and `operator=(const Matrix&)` copy **elements**. `ConstMatrixView` is the read-only counterpart a mutable view converts to implicitly.

Member	Description	Example
CONSTRUCTION & BINDING		
`MatrixView()` / `MatrixView(data, r, c, rs, cs)`	Empty view; view over caller storage with explicit strides.	`MatrixView<double>(p, 2, 2, 1)`
copy/move, `operator=`	Rebind to another view's storage (shallow).	`view = A.view();`
`operator=(const Matrix&)` / `copyFrom(v)`	Copy **elements** into the viewed storage (shapes must match).	`A.block(0,0,2,2) = B;`
ACCESS & SHAPE		
`operator()(i, j)` / `at(i, j)`	Unchecked / bounds-checked element access.	`view(0, 1)`
`rows` / `cols` / `rowStride` / `colStride`	Shape and strides.	`view.colStride()`
`isContiguous()` / `isEmpty()`	Dense-block test; empty test.	`view.isContiguous()`
`data()`	Pointer to element $(0, 0)$; honor strides unless contiguous.	`view.data()`
SUB-VIEWS		
`block(i, j, nr, nc)`	Rectangular sub-view.	`view.block(1, 1, 2, 2)`
`row(i)` / `col(j)` / `diagonal()`	$1 \times$cols, rows$\times 1$, and main-diagonal views.	`view.diagonal()`
`transposed()`	Shape/stride-swapped view; no elements move.	`view.transposed()`
BULK OPERATIONS		
`toMatrix()`	Deep-copy the elements into a fresh `Matrix`.	`view.toMatrix()`
`fill` / `setZero` / `scale(f)`	Write every element (mutable view only).	`view.scale(2.0);`
`swapWith(other)`	Exchange elements with another view.	`v1.swapWith(v2);`

```cpp
Matrix<double> A = Matrix<double>::Identity(4);
MatrixView<double> blk = A.block(1, 1, 2, 2);  // aliases A's storage
blk.fill(5.0);                                 // writes through to A
MatrixView<double> d = A.view().diagonal();    // 1 x 4 view of the diagonal
```

3.2 Operations and Primitives (`ops/`)

`Kernels` — the BLAS-shaped tuning surface

Level 1/2/3 routines as `static` members, following BLAS conventions: strided raw-pointer vectors, matrix operands as views (their strides playing the role of the leading dimension), and a `beta == 0` output treated as write-only. The level-3 routines are where blocking, packing, and the register-tiled micro-kernel (parameterized by `BlockSizes`) live, per the earlier discussion 1.1.

Member	Description	Example
LEVEL 1 — VECTOR/VECTOR, O(N)		
`scal(n, alpha, x, incx)`	$x \leftarrow \alpha x$.	`scal(n, 2.0, x, 1)`
`axpy(n, alpha, x, .., y, ..)`	$y \leftarrow \alpha x + y$.	`axpy(n, 1.0, x, 1, y, 1)`
`dot` / `dotc`	Unconjugated $\sum x_i y_i$ / conjugated $\sum \overline{x_i} y_i$.	`dot(n, x, 1, y, 1)`
`nrm2` / `asum`	2-norm (overflow-safe) / $\sum \lvert x_i \rvert$.	`nrm2(n, x, 1)`
`iamax`	Index of the largest-magnitude element.	`iamax(n, x, 1)`
`swap` / `copy`	Exchange / copy two strided vectors.	`copy(n, x, 1, y, 1)`
LEVEL 2 — MATRIX/VECTOR, O(N²)		
`gemv(trans, ..)`	$y \leftarrow \alpha\ \mathrm{op}(A)x + \beta y$.	`gemv(None, 1.0, A, x, 1, 0.0, y, 1)`
`ger(..)`	Rank-one update $A \leftarrow \alpha x y^T + A$.	`ger(1.0, x, 1, y, 1, A)`
`trsv(uplo, trans, diag, ..)`	Triangular solve $x \leftarrow \mathrm{op}(A)^{-1}x$.	`trsv(Upper, None, NonUnit, A, x, 1)`
`symv(uplo, ..)`	Symmetric $y \leftarrow \alpha A x + \beta y$.	`symv(Lower, 1.0, A, x, 1, 0.0, y, 1)`
LEVEL 3 — MATRIX/MATRIX, O(N³)		
`gemm(transA, transB, ..)`	$C \leftarrow \alpha\ \mathrm{op}(A)\ \mathrm{op}(B) + \beta C$.	`gemm(None, None, 1.0, A, B, 0.0, C)`
`syrk(uplo, trans, ..)`	Symmetric rank-k update $C \leftarrow \alpha\ \mathrm{op}(A)\mathrm{op}(A)^H + \beta C$.	`syrk(Upper, None, 1.0, A, 0.0, C)`
`trsm(uplo, trans, diag, ..)`	Triangular solve, many right-hand sides.	`trsm(Left, None, NonUnit, 1.0, A, B)`
BLOCKED-GEMM INTERNALS (BENCHMARKING)		
`tunedBlockSizes()`	The `BlockSizes` chosen for the current scalar.	`Kernels<double>::tunedBlockSizes()`
`packPanelA` / `packPanelB`	Pack a panel into a contiguous, kernel-friendly layout.	`packPanelA(a, buffer)`
`microKernel(kc, ..)`	The $m_r \times n_r$ rank-k_c register-tile update.	`microKernel(kc, pa, pb, C)`

```
// C <- 1.0 * A * B + 0.0 * C   (beta == 0: C is write-only)
Matrix<double> C(A.rows(), B.cols());
Kernels<double>::gemm(Transposition::Kind::None, Transposition::Kind::None,
                      1.0, A.view(), B.view(), 0.0, C.view());
```

`Norm` — vector and matrix norms

Static routines separating the cheap entrywise/induced norms from the $O(n^3)$ SVD-based ones, plus the diagnostics the correctness suite leans on. The scaled 2-norm avoids an intermediate $\sum |x_i|^2$ that would overflow for large entries — the same concern that shapes `Givens::FromPair`.

Member	Description	Example
VECTOR NORMS		
`vectorOne` / `vectorInfinity`	$\sum \lvert x_i \rvert$ / $\max \lvert x_i \rvert$.	`Norm<double>::vectorOne(x)`

Member	Description	Example
`vectorTwo` / `vectorTwoScaled`	Euclidean norm; the overflow-safe scaled variant.	`Norm<double>::vectorTwoScaled(x)`
`vectorP(x, p)`	General p-norm.	`Norm<double>::vectorP(x, 3.0)`
MATRIX NORMS		
`matrixOne` / `matrixInfinity`	Max absolute column / row sum.	`Norm<double>::matrixOne(A)`
`matrixFrobenius` / `matrixMax`	Frobenius; largest entry magnitude.	`Norm<double>::matrixFrobenius(A)`
`matrixTwo` / `matrixNuclear`	Spectral / nuclear norm (SVD; $O(n^3)$).	`Norm<double>::matrixTwo(A)`
DIAGNOSTICS		
`distance(x, y)` / `relativeError(approx, exact)`	$\|x-y\|$; $\|\text{approx}-\text{exact}\|/\|\text{exact}\|$.	`Norm<double>::distance(x, y)`
`residualNorm(A, x, b)`	$\|b-Ax\|$.	`Norm<double>::residualNorm(A, x, b)`
`backwardError(A, x, b)`	$\|b-Ax\|/(\|A\|\|x\|+\|b\|)$.	`Norm<double>::backwardError(A, x, b)`
`orthogonalityDefect(Q)`	$\|Q^HQ-I\|_F$.	`Norm<double>::orthogonalityDefect(Q)`

```
double bwd = Norm<double>::backwardError(A, x, b);
// bwd ~ machine epsilon means x exactly solves a nearby system,
// regardless of how ill-conditioned A is.
```

`Householder` — reflectors

A reflector $H = I - \beta vv^H$ stored implicitly as its essential vector (the leading 1 of v is omitted) and a real β. `FromVector` / `FromColumn` build the reflector mapping x to $\pm\|x\|e_1$, choosing the sign *opposite* x_1 (opposite its phase for complex `T`) so the leading subtraction cannot cancel. Application is a two-pass $O(mn)$ update that never forms H.

Member	Description	Example
HOUSEHOLDER — SINGLE REFLECTOR		
`Householder()` / `Householder(essential, beta)`	Identity reflector; construct from stored parts.	`Householder<double>(tail, beta)`
`FromVector(x)` / `FromColumn(a, col, startRow)`	*(static)* Build H zeroing all but the first entry.	`Householder<double>::FromVector(x)`
`essential()` / `beta()` / `size()` / `isIdentity()`	Stored tail, coefficient, dimension, no-op test.	`h.beta()`
`applyLeft` / `applyRight` / `applyLeftConjugate`	HA, AH, H^HA in place (H never formed).	`h.applyLeft(A.view())`
`apply(x)`	Reflect a vector in place.	`h.apply(x)` $\to (-5,0,0)$
`toMatrix(n)`	Dense H embedded in an identity (tests).	`h.toMatrix(3)`
HOUSEHOLDERSEQUENCE — AGGREGATED Q		
`HouseholderSequence()` / `(reflectors, betas)`	Empty; or from packed reflector columns and betas.	`HouseholderSequence<double>(V, betas)`
`append(h)` / `count()`	Add a reflector; how many are held.	`seq.append(h);`

Member	Description	Example
`applyLeft` / `applyRight` / `applyLeftTranspose`	QA, AQ, $Q^H A$, applied through level-3 kernels.	`seq.applyLeftTranspose(A.view())`
`toMatrix(n)` / `firstColumns(n, k)`	Full Q; its leading k columns (thin Q).	`seq.firstColumns(5, 3)`
`blockV` / `blockT` / `buildBlockRepresentation(bs)`	Compact-WY factors $Q = I - VTV^H$ and their (re)build.	`seq.buildBlockRepresentation(32)`

```
Vector<double> x{3.0, 4.0, 0.0};
Householder<double> h = Householder<double>::FromVector(x);
h.apply(x);             // x -> (-5, 0, 0): the column is reflected onto e_1
```

`Givens` — plane rotations

The other orthogonal primitive, whose full per-method reference and worked example are in the worked-kernel section 2: a unitary 2×2 rotation stored as (c, s) plus its plane indices, used to zero one entry at a time. `GivensSequence` is the ordered product of such rotations — the accumulated sweep of a Jacobi eigenvalue pass or a bidiagonal chase.

4 Design

Headers declare, source files define. Every class template is explicitly instantiated in `src/` for the supported scalar list, so the project builds as an ordinary compiled library with real separate compilation instead of a header-only template dump. Unimplemented members throw a `LinalgError` naming themselves, which makes the remaining work greppable and turns the test suite into a red-to-green progress meter.

> **Remark (Progress at a glance) .** Every stub names itself when it throws, so the remaining work is one grep away — the per-file count doubles as a burndown chart:
>
> ```
> grep -rc "not implemented" src/ | sort -t: -k2 -rn
> ```

Correctness is checked against identities rather than golden files: $PA = LU$, $Q^H Q = I$, residual and backward-error bounds sized by machine epsilon and the condition number, ill-conditioned stress inputs like the Hilbert matrix.

5 Documentation

The project is documented at three levels, each aimed at a different reader and kept close to the thing it describes.

- **API reference — in the headers.** Every declaration in `include/` carries a Doxygen-style comment block (`@brief` , `@param` , `@return` , `@throws`), so the headers double as the reference manual: the contract for a function sits directly above its signature, where it is hardest to let drift out of sync with the code. Because the headers are declaration-only, reading one is a tour of *what* a component offers without the noise of *how* it is implemented.
- **Mathematical notes — the "why".** A companion set of notes derives the algorithms and the rationale behind their computational choices — overflow-safe scaling, sign choices that avoid

cancellation, deflation criteria — at a depth the source comments deliberately leave out. They are linked from the top of this document and cross-referenced from the code where a derivation is load-bearing (see, for instance, the `dlartg` reference in `Givens::FromPair` and the structured-decomposition note in `Givens::inverse`).

- **This report — the design narrative.** The high-level story of how the tiers fit together, why the kernels are the single tuning surface, and how a representative kernel maps to its math. It is written in Typst and lives beside the source it describes:

  ```
  typst compile docs/main.typ
  ```

> **Remark (One convention, three views) .** The three layers share a single organizing idea — *the header layout is the table of contents.* The directory tiers of the architecture diagram 1 name the API sections, the module reference walks those same files, and the notes are indexed by the same components. A reader can enter at any level and cross to the others without relearning the map.

6 Directions for Expansion

Expanding this library to include methods of large-scale and structured computation is the natural next step, in rough order of ambition:

- **Iterative Krylov methods** (already scaffolded): conjugate gradients, GMRES with restarting, BiCGSTAB, and LSQR, with Jacobi, SSOR, and incomplete-Cholesky preconditioners — the bridge from dense $O(n^3)$ factorizations to problems where only matrix–vector products are affordable.
- **Iterative eigensolvers:** power iteration, Lanczos with selective reorthogonalization, and implicitly restarted Arnoldi for a few eigenpairs of large matrices.
- **Performance engineering:** cache-blocked and packed `gemm` with a register-tiled micro-kernel, blocked variants of LU/Cholesky/QR, and compact-WY application of reflector sequences, benchmarked against the naive implementations the correctness tests already validate.
- **Sparse storage and operations,** which the iterative methods are deliberately shaped to exploit (they touch the matrix only through products and preconditioner applications).

7 Building

```
cmake -B build -DLINALG_BUILD_TESTS=ON
cmake --build build
ctest --test-dir build
```